ARTIFICIAL INTELLIGENCE

Complete Exam Revision Guide & Detailed Syllabus Explainer

Course Code: CS701 (B.Tech CSE)	**Department:** Computer Science & Engineering
University: Assam Don Bosco University (ADBU)	**Instructor / Source:** Prof. Alok Choudhury & Prof. R.C. Chakraborty
Coverage: All 8 Lecture Modules & PDFs	**Includes:** 36 Core Definitions, 24 Official Diagrams, Proofs & 20Q Quiz

EXAM REVISION GUIDE EXECUTIVE SUMMARY

This master document provides a consolidated, in-depth mathematical and conceptual explanation of the entire Artificial Intelligence curriculum. It integrates official textbook diagrams, formal algorithms, complexity analyses, the PEAS agent matrix, heuristic search principles, logic proof techniques, and a fully resolved 20-question exam readiness test.

Section	Topic & Lecture Focus	Primary Course Reference
Module 1	Foundations, DIKW Hierarchy & 8 Eras of AI History	1_Introduction_Agent.pdf
Module 2	Intelligent Agents, PEAS Framework & 5 Architectures	2_Agent Architecture and environment.pdf
Module 3	Problem Solving & Informed Search (Greedy, A*, Consistency)	4_ Informed Search Algorithms.pdf
Module 4	Memory-Bounded Heuristic Search (IDA*, RBFS, SMA*)	5_Memory Bounded Search.pdf
Module 5	Knowledge Representation, Production Systems & Prolog	1_Knowledge_Representations.pdf
Module 6	Formal Logic, Rules of Inference & 8 Proof Methods	4_Logic.pdf & 2_Rules-of-inference.pdf
Module 7	Master AI Glossary (36 High-Yield Exam Definitions)	Consolidated Course Glossary
Module 8	20-Question Comprehensive Exam Simulation & Answer Key	Exam Readiness Assessment

Module 1: Foundations, DIKW Pyramid & History of AI

Artificial Intelligence is defined through two foundational paradigms: the scientific pursuit of understanding and modeling cognitive processes, and the engineering endeavor of constructing rational computational agents capable of sensing, deliberating, and acting.

THE FOUR DIMENSIONS OF ARTIFICIAL INTELLIGENCE

The 4 Perspectives of AI (Russell & Norvig):

- **Thinking Humanly:** Cognitive science approach; empirical modeling of the human mind.
- **Acting Humanly:** Turing Test approach (1950); passing behavioral benchmarks in NLP, reasoning, knowledge, and ML.
- **Thinking Rationally:** Laws of Thought approach; rigorous deductive inference via formal syllogisms and logic.
- **Acting Rationally:** Rational Agent approach; selecting actions that maximize expected performance given available percepts.

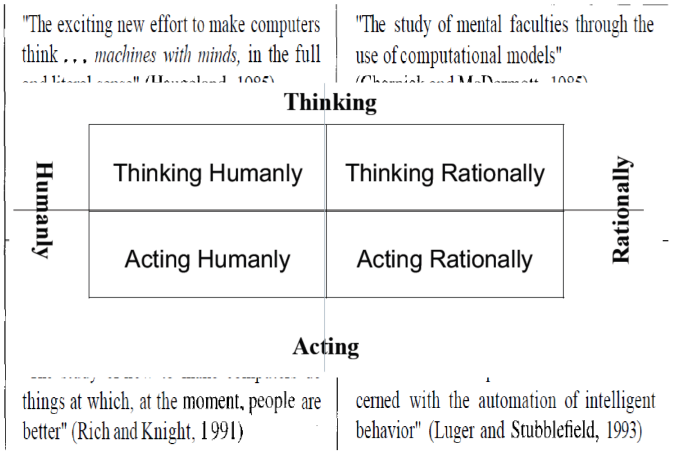


Figure: *The Four Approaches to AI Matrix — [Source: PDF 1, Slide 10]*

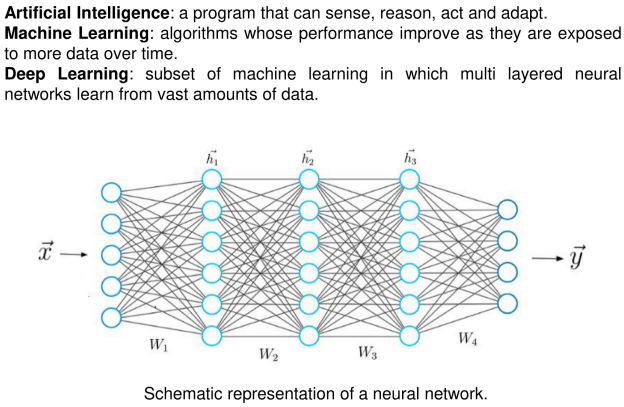


Figure: *Sense, Reason, Act Framework Loop — [Source: PDF 1, Slide 7]*

The DIKW Hierarchy

A foundational model in knowledge systems conceptualizing the progressive enrichment of human and computational cognition:

1. **Data:** Disconnected, unorganized raw symbols, numbers, and facts without contextual framing.
2. **Information:** Structured data enriched with relational context and meaning (answers *who, what, where*).
3. **Knowledge:** Understanding of patterns, associations, rules, and procedures; encapsulated as *'knowing that'* (declarative) and *'knowing how'* (procedural).
4. **Wisdom / Intelligence:** Evaluating principles and rationale (answers *why*).

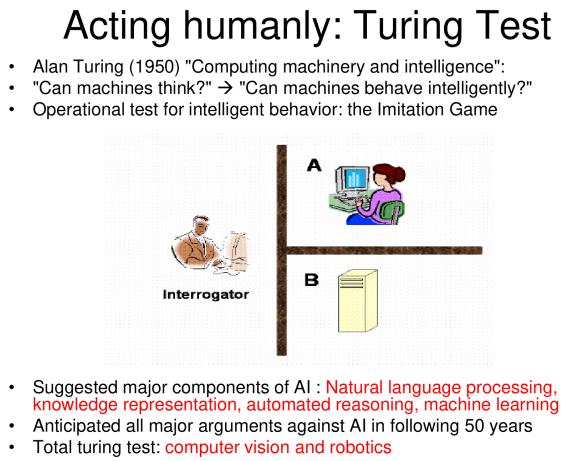


Figure: *Alan Turing's 1950 Imitation Game (Turing Test) Setup — [Source: PDF 1, Slide 11]*

Key Historical Eras & Milestones in AI

- **1943–1955 (Gestation):** McCulloch & Pitts propose the first mathematical model of artificial neurons. Alan Turing publishes *'Computing Machinery and Intelligence'*.
- **1956 (Birth of AI):** Dartmouth College Workshop. John McCarthy, Marvin Minsky, Claude Shannon, and Nathaniel Rochester officially coin the term *'Artificial Intelligence'*.
- **1952–1969 (Early Enthusiasm):** Newell & Simon build the Logic Theorist and General Problem Solver (GPS). Arthur Samuel constructs self-learning checkers programs.
- **1966–1973 (Reality Check & AI Winter I):** Early microworlds fail against combinatorial explosion. Minsky & Papert's *Perceptrons* book proves single-layer neural nets cannot solve XOR.
- **1969–1979 (Knowledge-Based / Expert Systems):** The shift toward domain-specific knowledge. DENDRAL analyzes molecular mass spectra; MYCIN diagnoses bacterial infections (~600 rules).
- **1980–1988 (Commercialization & Fifth Generation):** Commercial expert systems boom (DEC XCON saves millions). Japan invests in the massive Fifth Generation Project (Prolog).
- **1986–Present (Neural Renaissance):** Rumelhart, Hinton, and Williams popularize Backpropagation for multi-layer perceptrons, overcoming the XOR limitation.
- **2012–Present (Deep Learning & Foundation Models):** AlexNet wins ImageNet (2012); AlphaGo beats world champions (2016); Transformers and Large Language Models achieve general reasoning milestones.

Module 2: Intelligent Agents & The PEAS Framework

An **Agent** is formally defined as an entity that perceives its environment through **Sensors** and acts upon it via **Actuators**. The mathematical relation is encapsulated by: **Agent = Architecture + Program**. The abstract mathematical mapping from complete percept histories to actions is the **Agent Function** $f: P^* \to A$.

RATIONAL AGENT PRINCIPLE

Rationality vs. Omniscience:
A **Rational Agent** chooses an action expected to maximize its performance measure, based on: (1) Percept sequence to date, (2) Built-in domain knowledge, and (3) Actions available.
Critical Exam Distinction: Rationality is NOT omniscience. Omniscience implies knowing the actual outcome in advance, which is impossible in real-world stochastic environments.

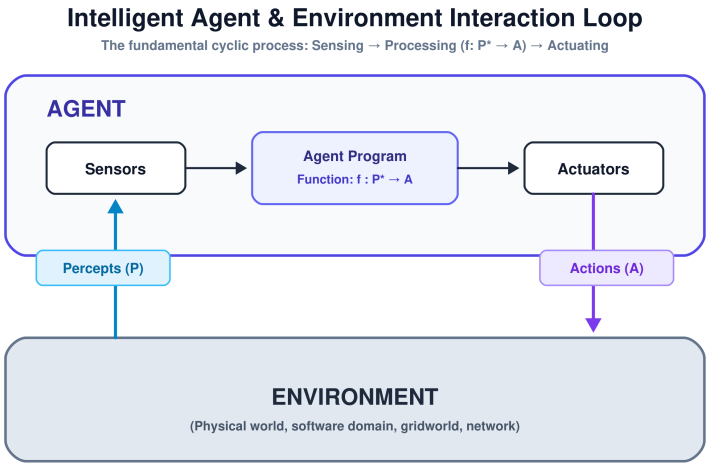


Figure: *Official Agent-Environment Interaction Model — [Source: PDF 2, Slide 6]*

The PEAS Specification Matrix

Before designing an agent, the task environment must be rigorously specified using the PEAS framework: **Performance measure** (P), **Environment** (E), **Actuators** (A), and **Sensors** (S).

Agent	Performance Measure (P)	Environment (E)	Actuators (A)	Sensors (S)
Autonomous Taxi	Safety, speed, legal compliance, comfort, profit	Roads, traffic, pedestrians, weather	Steering, accelerator, brakes, horn, turn signal	Cameras, LiDAR, radar, GPS, speedometer
Medical Diagnostician	Healthy patient, minimized cost, diagnostic accuracy	Patient, clinical staff, laboratory	Prescriptions, test orders, screen reports	Keyboard entry of symptoms, vitals, lab data
Satellite Image Analysis	Image classification accuracy, discovery rate	Downlink telemetry, orbit paths, cloud cover	Antenna repointing, alert dispatches	Multi-spectral optical & infrared sensors
Part-Picking Robot	Sorting percentage, sorting speed, zero damage	Conveyor belt with mixed parts, bins	Jointed robotic arm, suction gripper, motor	Stereo cameras, grip pressure sensors
Smart Vacuum Cleaner	Cleanliness score, battery life, room coverage	Floors, carpet, obstacles, stairs, pets	Drive wheels, brushes, suction motor	Infrared cliff sensor, bump switch, dirt sensor

The 7 Environment Dimensions (Properties)

Dimension	Contrasting Extremes	Formal Definition & Exam Example
1. Observability	Fully vs. Partially	**Fully:** Complete state access (Chess). **Partially:** Sensor noise or blind spots require internal world model (Poker, Driving).
2. Determinism	Deterministic vs. Stochastic	**Deterministic:** Next state strictly fixed by state + action (Chess). **Stochastic:** Probabilities/noise exist (Backgammon, Driving).
3. Episodic	Episodic vs. Sequential	**Episodic:** Current action has no effect on future episodes (Defect inspection). **Sequential:** Actions cascade into future (Chess, Driving).
4. Dynamism	Static vs. Dynamic vs. Semi	**Static:** World doesn't change during deliberation (Crosswords). **Dynamic:** Changes constantly (Driving). **Semi:** Score decays with time (Timed Chess).
5. Continuity	Discrete vs. Continuous	**Discrete:** Countable finite states and actions (Chess). **Continuous:** Real-valued continuous coordinates, speed, angles (Driving).
6. Agent Count	Single vs. Multi-Agent	**Single:** Solo puzzle solving (Solitaire). **Multi-Agent:** Competitive (zero-sum, Chess) or Cooperative (fleet navigation).
7. Known Physics	Known vs. Unknown	**Known:** Environment rules are given upfront. **Unknown:** Agent must explore and discover transition physics first.

The 5 Agent Architectures & Schematics

There are five fundamental agent program architectures ranging from simple condition-action lookup to full adaptive learning:

1. Simple Reflex Agent

Selects actions based exclusively on the current percept, completely ignoring percept history. Operates via direct condition-action rules: IF condition THEN action. Fails completely in partially observable environments, frequently getting trapped in infinite loops.

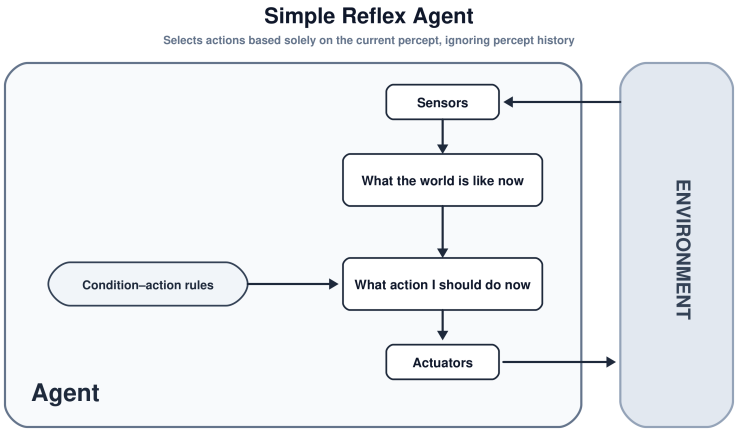


Figure: *Simple Reflex Agent Architecture — [Source: PDF 2, Slide 11]*

2. Model-Based Reflex Agent

Overcomes partial observability by maintaining an **internal state**. It relies on two types of knowledge: (1) *Transition Model:* how the world evolves independently of the agent, and (2) *Sensor Model:* how the agent's actions alter the world.

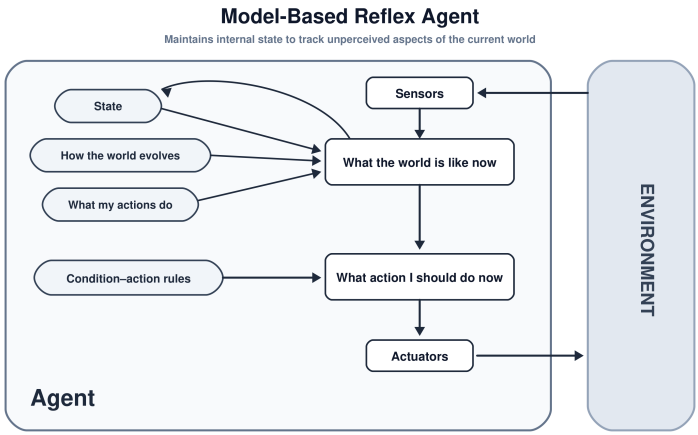


Figure: *Model-Based Reflex Agent Architecture — [Source: PDF 2, Slide 13]*

3. Goal-Based Agent

Combines internal state knowledge with explicit **goal descriptions** indicating desirable situations. Employs Search and Planning algorithms to compute a sequence of actions that achieves the goal.

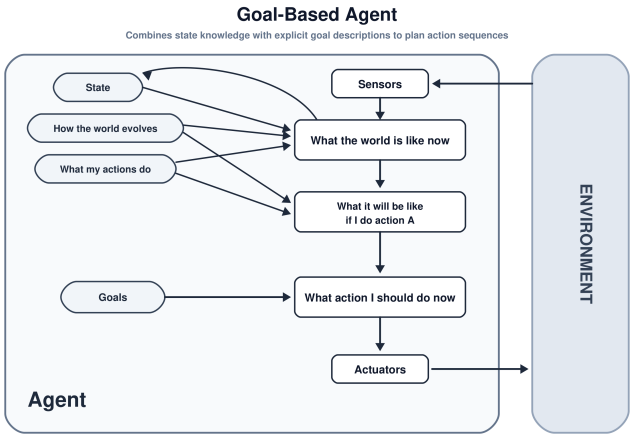


Figure: *Goal-Based Agent Architecture — [Source: PDF 2, Slide 15]*

4. Utility-Based Agent

Goals alone provide only a binary pass/fail assessment. A **utility function** $U(s)$ maps world states to real numbers, scoring degrees of happiness or preference. Allows the agent to make rational trade-offs between competing goals (e.g., speed vs. safety).

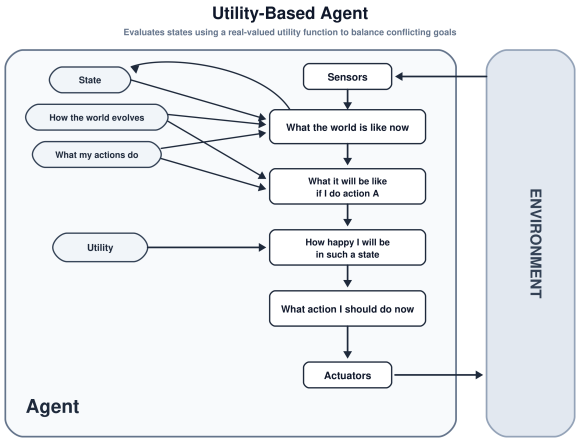


Figure: *Utility-Based Agent Architecture — [Source: PDF 2, Slide 17]*

5. Learning Agent

Separated into four distinct functional modules:

- **Learning Element:** Responsible for making improvements based on past experiences.
- **Performance Element:** Responsible for selecting external actions (equivalent to previous agents).
- **Critic:** Evaluates the agent's behavior against an external performance standard.
- **Problem Generator:** Suggests exploratory actions leading to new, informative experiences (exploration vs. exploitation).

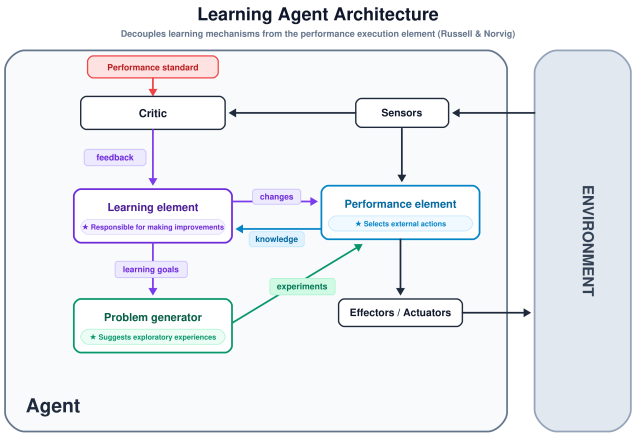


Figure: *Learning Agent Architecture — [Source: PDF 2, Slide 19]*

Module 3: Problem Solving & Informed / Heuristic Search

A search problem is formally formulated by 5 components: (1) **Initial State** s_0, (2) **Actions** $A(s)$, (3) **Transition Model** $\text{Result}(s, a)$, (4) **Goal Test** $G(s)$, and (5) **Path Cost** $c(s, a, s')$. Search algorithms are evaluated along 4 metrics: **Completeness**, **Time Complexity**, **Space Complexity**, and **Optimality**.

HEURISTIC PROPERTIES & OPTIMALITY CRITERIA

Heuristic Function & Mathematical Guarantees:

- A **Heuristic** $h(n)$ estimates the cost of the cheapest path from node n to the goal (derived via problem relaxations).
- **Admissibility:** $0 \le h(n) \le h^*(n)$ (never overestimates true cost). Guarantees optimality in A^* tree search.
- **Consistency (Monotonicity):** $h(n) \le c(n, a, n') + h(n')$ (triangle inequality). Guarantees optimality in A^* graph search.
- *Theorem:* **Consistency $\implies$ Admissibility**, but admissibility does NOT always imply consistency!

Comparison of Search Cost Evaluation Functions

Algorithm	Evaluation Function	Completeness	Optimality	Complexity / Flaw
Uniform Cost (UCS)	f(n) = g(n)	Yes (step cost ≥ ε)	Yes	Expands circular contours in all directions; slow.
Greedy Best-First	f(n) = h(n)	No (can loop in graph)	No	Chases heuristic; gets fooled by deceptive paths.
A* Search	f(n) = g(n) + h(n)	Yes	Yes (if admissible)	Memory explosion: Space complexity is O(b^d).

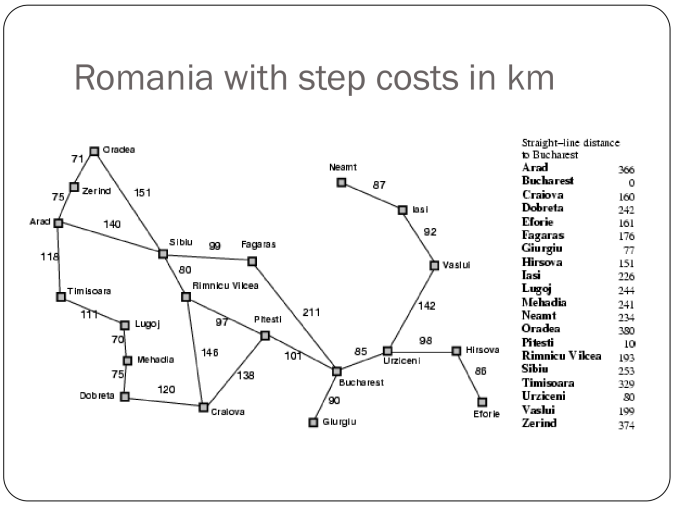


Figure: *The Romania Route Benchmark Problem Map — [Source: PDF 4, Slide 17]*

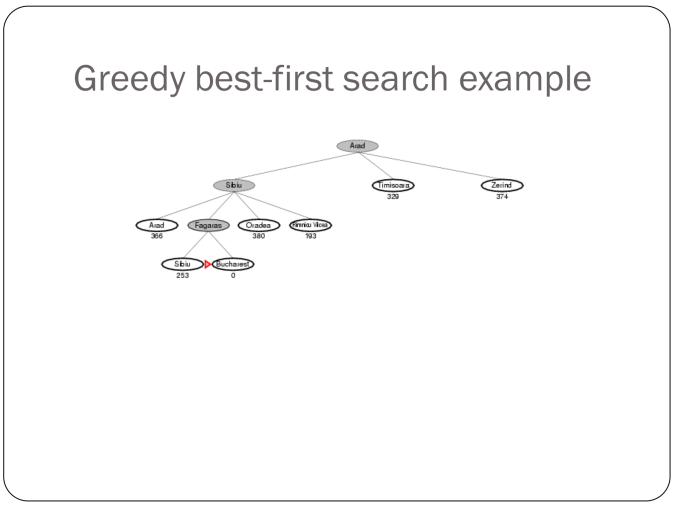


Figure: *Greedy Best-First Search on Romania Tree — [Source: PDF 4, Slide 22]*

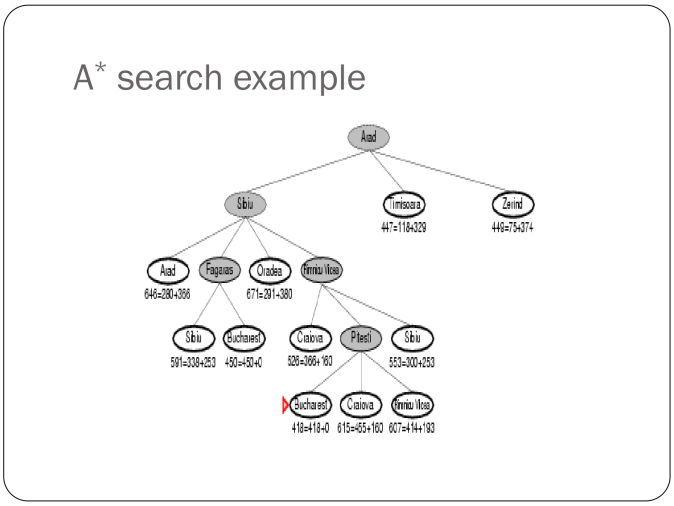


Figure: *A* Search on Romania Tree — [Source: PDF 4, Slide 30]*

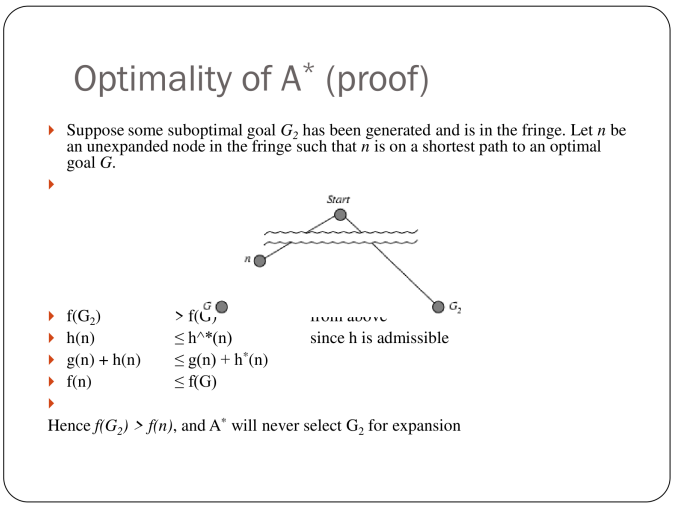


Figure: *A* Optimality Proof via Concentric f-Cost Contours — [Source: PDF 4, Slide 33]*

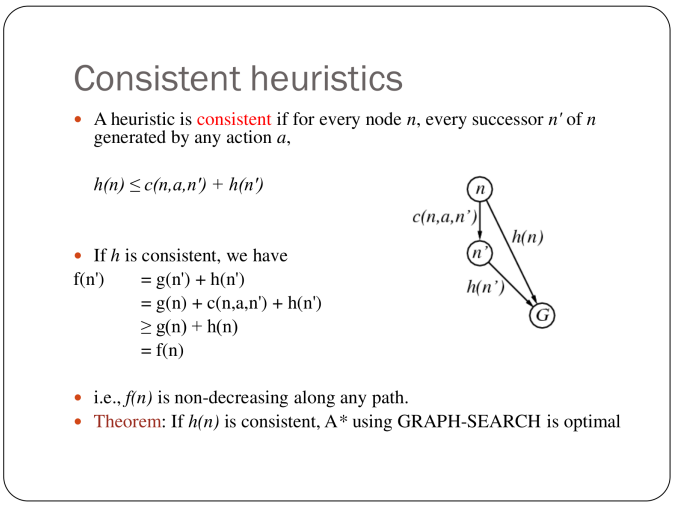


Figure: *Consistency / Monotonicity Triangle Inequality — [Source: PDF 4, Slide 34]*

Module 4: Memory-Bounded Heuristic Search

While A^* search is optimally efficient in terms of expanded nodes, its space complexity is exponential: $O(b^d)$. Every generated state must be preserved in either the Open or Closed list. For a problem with branching factor $b=10$ and depth $d=10$, A^* generates over 10^{10} nodes, demanding over 100 Gigabytes of RAM and exhausting physical memory within minutes. Memory-bounded algorithms resolve this space bottleneck.

MEMORY-BOUNDED PARADIGMS

Core Memory-Bounded Algorithms:

• **IDA* (Iterative Deepening A*):** Uses depth-first search bounded by an f-cost limit rather than depth. Requires only linear space $O(b \cdot d)$.

• **RBFS (Recursive Best-First Search):** Mimics best-first search in linear memory $O(b \cdot d)$. Keeps track of the best alternative f-cost at each ancestor node.

• **SMA* (Simplified Memory-Bounded A*):** Utilizes all allocated memory buffer M. When memory is full, it evicts the *shallowest, highest f-cost* leaf node and stores its backed-up value in its parent.

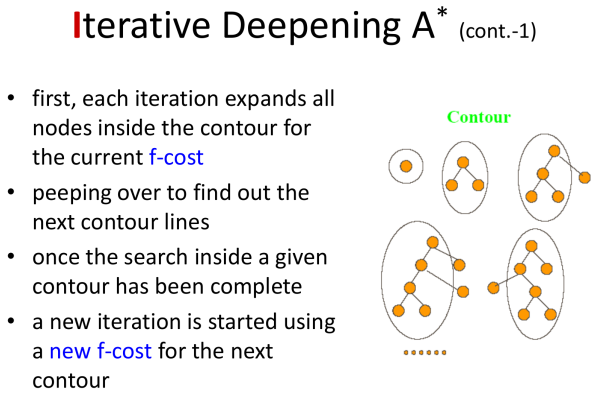


Figure: *IDA* Contour Search Iterations & f-Limit Thresholds — [Source: PDF 5, Slide 4]*

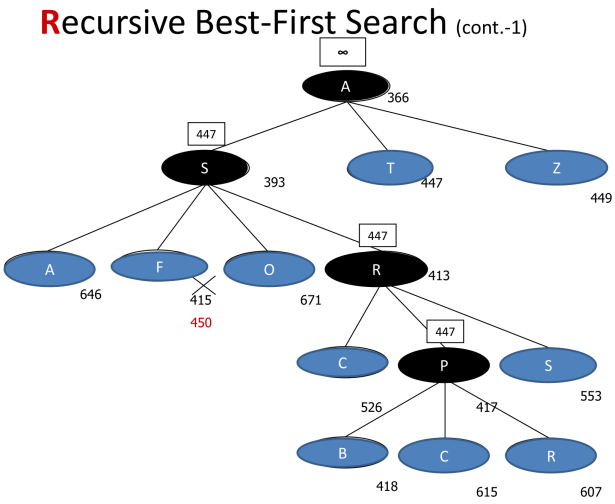


Figure: *RBFS Search Progression on Romania with Backed-Up Values — [Source: PDF 5, Slide 8]*

SMA* Progression & Eviction Mechanics

SMA* operates by expanding the newest best leaf. When memory buffer M is exhausted:
1. It finds the shallowest node with the highest f-value (the worst leaf) and removes it from memory.
2. It backs up the forgotten leaf's f-cost into its parent node.
3. If all other alternative branches prove worse, the parent node can regenerate the forgotten child.
• **Completeness:** Guaranteed if memory buffer $M \ge d$ (shallowest goal depth fits in memory).
• **Optimality:** Guaranteed if memory buffer $M \ge d^*$ (optimal path fits in memory).

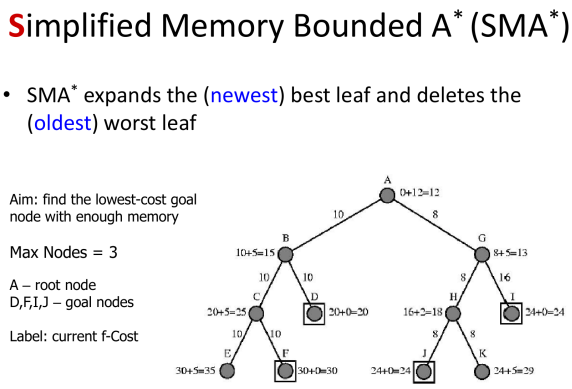


Figure: *SMA* Stage 1: Filling Allocated Memory Buffer M — [Source: PDF 5, Slide 11]*

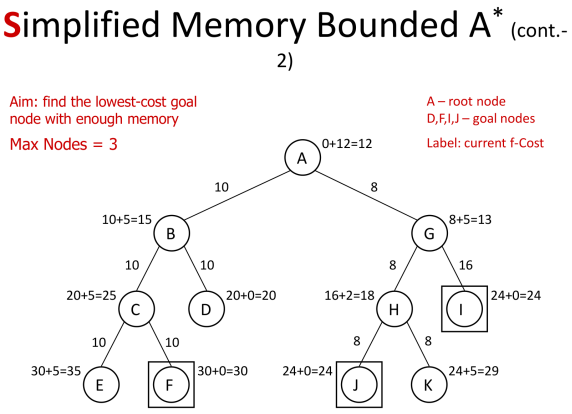


Figure: *SMA* Stage 2: Evicting Worst Leaf & Backing Up Value — [Source: PDF 5, Slide 13]*

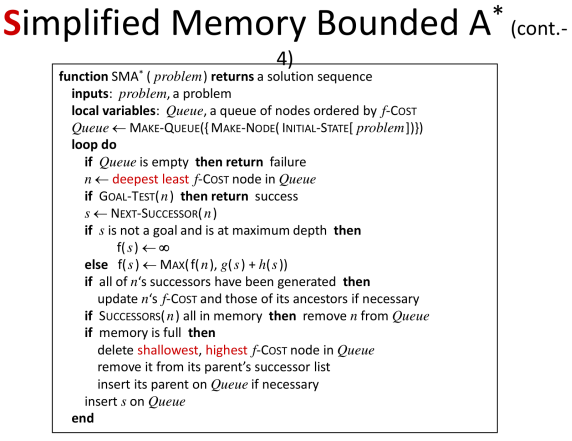


Figure: *SMA* Stage 3: Regenerating Leaf Node from Parent Backup — [Source: PDF 5, Slide 15]*

Memory-Bounded Complexity Matrix

Algorithm	Time Complexity	Space Complexity	Completeness	Optimality
A* Search	O(b^d)	O(b^d) [Exponential]	Yes	Yes (admissible h)
IDA*	O(b^d) [worse with fine costs]	O(b · d) [Linear]	Yes	Yes
RBFS	O(b^d) [can thrash]	O(b · d) [Linear]	Yes	Yes
SMA*	O(b^d)	O(M) [Exact buffer cap]	If M ≥ d	If M ≥ d*

Module 5: Knowledge Representation & Production Systems

A Knowledge Representation (KR) system must satisfy 4 primary criteria: **Representational Adequacy** (capturing all domain concepts), **Inferential Adequacy** (manipulating structures to derive new facts), **Inferential Efficiency** (focusing reasoning in fruitful directions), and **Acquisitional Efficiency** (ease of integrating new knowledge).

KNOWLEDGE REPRESENTATION SCHEMES

Knowledge Schemes & Typology:

- **Declarative ('Knowing That'):** Static facts, relationships, and axioms (e.g. Prolog facts, SQL tables).
- **Procedural ('Knowing How'):** Step-by-step algorithms, control instructions, and rules of execution.
- **Inheritable Knowledge:** Graphical structures (Semantic Nets & Frames) where properties propagate down IsA hierarchies.
- **Production Systems:** Composed of *Working Memory* (facts), *Rule Base* (IF-THEN rules), and an *Inference Engine*.

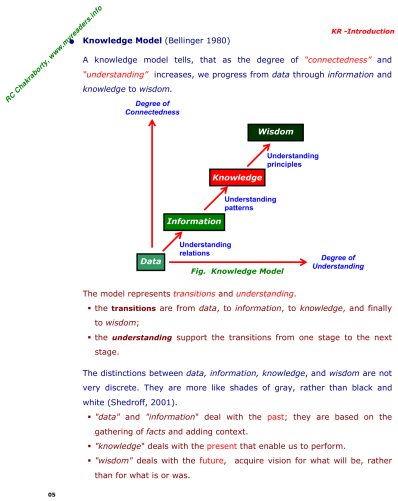


Figure: *The DIKW Knowledge Model (Bellinger 1980) — [Source: PDF KR, Slide 5]*

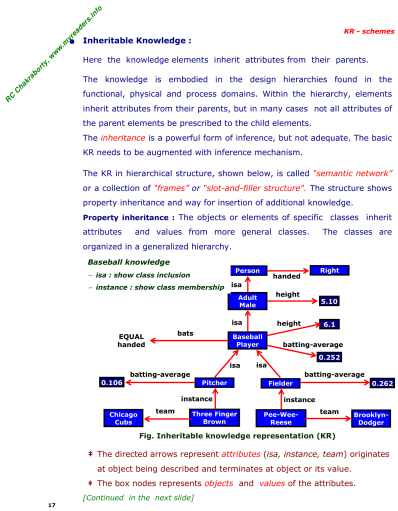


Figure: *Inheritable Semantic Network Hierarchy — [Source: PDF KR, Slide 17]*

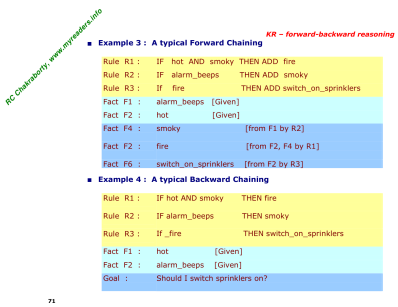


Figure: *Production System Architecture (Working Memory + Rules + Engine) — [Source: PDF KR, Slide 71]*

Forward Chaining vs. Backward Chaining

• **Forward Chaining (Data-Driven):** Starts with known facts in Working Memory, checks rule conditions, fires matching rules, and asserts new facts into memory until a goal is proven. Uses *Conflict Resolution* strategies:
- **Refractory:** A rule cannot fire twice on the identical piece of data (prevents infinite loops).
- **Recency:** Prefers rules matching newly generated facts (time-tags).
- **Specificity:** Prefers rules having more detailed condition clauses.
• **Backward Chaining (Goal-Driven):** Starts with a hypothesis (goal), finds rules concluding it, and recursively sets unproven premises as new sub-goals.

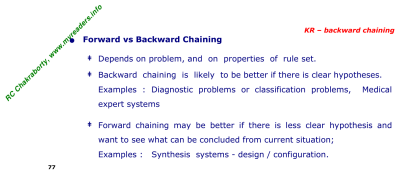


Figure: *Forward vs. Backward Chaining Comparison Table — [Source: PDF KR, Slide 77]*

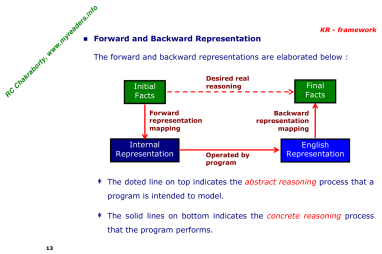


Figure: *Facts ↔ Representation Mapping — [Source: PDF KR, Slide 13]*

Logic Programming & Prolog Fundamentals

Based on Robert Kowalski's landmark formulation: **Algorithm = Logic + Control**.

• **Facts (Unit Clauses):** parent(tom, bob)., female(pam). (end with period).

• **Rules (Horn Clauses):** grandparent(X, Z) :- parent(X, Y), parent(Y, Z). ('Head is true IF Body is true').

• **Queries:** ?- grandparent(tom, Who). unifies variables (capitalized) using SLD resolution and depth-first backtracking.

Module 6: Logic, Rules of Inference & Proof Methods

Propositional logic studies declarative statements with boolean truth values. Compound statements are formed with connectives $\neg, \land, \lor, \to, \leftrightarrow$. A **Tautology** is always true; a **Contradiction** is always false; a **Contingency** varies. Entailment $\alpha \models \beta$ means in every model where α is true, β is also true.

The 8 Propositional Rules of Inference

Rule of Inference	Premises & Derivation	Formal Name & Plain English Meaning
Modus Ponens	p, p → q ■ q	Affirming antecedent: If p is true, and p implies q, then q must be true.
Modus Tollens	¬q, p → q ■ ¬p	Denying consequent: If p implies q, and q is false, then p must be false.
Hypothetical Syllogism	p → q, q → r ■ p → r	Transitivity: If p implies q, and q implies r, then p implies r.
Disjunctive Syllogism	p ∨ q, ¬p ■ q	Elimination: If p or q is true, and p is false, then q must be true.
Addition	p ■ p ∨ q	Disjunction introduction: If p is true, then p OR anything else is true.
Simplification	p ∧ q ■ p	Conjunction elimination: If both p and q are true, then p alone is true.
Conjunction	p, q ■ p ∧ q	Conjunction introduction: If p is true and q is true, both are true together.
Resolution	p ∨ q, ¬p ∨ r ■ q ∨ r	Resolvent derivation: Key automated theorem proving & Prolog rule.

Quantifier Rules of Inference

- **Universal Instantiation (UI):** $\forall x P(x) \vdash P(c)$ for any arbitrary constant c.
- **Universal Generalization (UG):** $P(c)$ for an arbitrary element $c \vdash \forall x P(x)$.
- **Existential Instantiation (EI):** $\exists x P(x) \vdash P(c)$ for some newly introduced specific element c.
- **Existential Generalization (EG):** $P(c)$ for a known witness $c \vdash \exists x P(x)$.
- **Universal Modus Ponens:** $\forall x (P(x) \to Q(x)), P(a) \vdash Q(a)$.

The 8 Mathematical Proof Techniques

Proof Method	Mechanism / Logical Blueprint	Standard Exam Application
1. Direct Proof	Assume p is true $\to$ deduce q via axioms and definitions.	If n is odd, n^2 is odd: $(2k+1)^2 = 4k^2+4k+1 = 2(2k^2+2k)+1$.
2. Contraposition	To prove $p \to q$, prove logically equivalent $\neg q \to \neg p$.	If $3n+2$ is odd, then n is odd: assume $n=2k$ is even $\implies 6k+2$ is even.
3. Contradiction	Assume $\neg p$ is true $\to$ derive an impossible contradiction F.	Proving $\sqrt{2}$ is irrational, or that the set of prime numbers is infinite.
4. Proof by Cases	Break domain into exhaustive cases: $(p_1 \lor \dots \lor p_n) \to q$.	Proving $\|xy\| = \|x\|\|y\|$ by evaluating positive, negative, and zero cases.
5. Constructive	Find an explicit concrete witness c satisfying $P(c)$.	Taxicab number: $1729 = 10^3 + 9^3 = 12^3 + 1^3$.
6. Non-Constructive	Prove existence of witness via contradiction without producing it.	Proving irrational numbers a, b exist such that a^b is rational.
7. Counterexample	Disprove a universal claim $\forall x P(x)$ by finding one witness c with $\neg P(c)$.	Refuting 'all primes are odd' by naming the even prime 2.
8. Uniqueness Proof	Prove existence $P(x)$, then assume $P(y) \implies y = x$.	Proving a linear equation $ax + b = 0$ ($a \ne 0$) has exactly one root.

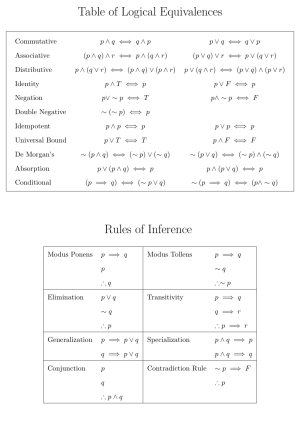


Table of Logical Equivalences

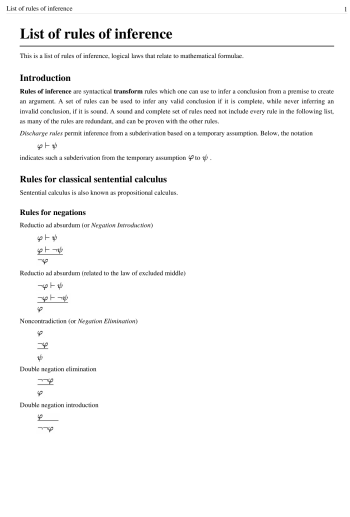
Commutative	$p \wedge q \iff q \wedge p$	$p \vee q \iff q \vee p$
Associative	$(p \wedge q) \wedge r \iff p \wedge (q \wedge r)$	$(p \vee q) \vee r \iff p \vee (q \vee r)$
Distributive	$p \wedge (q \vee r) \iff (p \wedge q) \vee (p \wedge r)$	$p \vee (q \wedge r) \iff (p \vee q) \wedge (p \vee r)$
Identity	$p \wedge T \iff p$	$p \vee F \iff p$
Negation	$p \vee \sim p \iff T$	$p \wedge \sim p \iff F$
Double Negative	$\sim (\sim p) \iff p$	
Idempotent	$p \wedge p \iff p$	$p \vee p \iff p$
Universal Bound	$p \vee T \iff T$	$p \wedge F \iff F$
De Morgan's	$\sim (p \wedge q) \iff (\sim p) \vee (\sim q)$	$\sim (p \vee q) \iff (\sim p) \wedge (\sim q)$
Absorption	$p \vee (p \wedge q) \iff p$	$p \wedge (p \vee q) \iff p$
Conditional	$(p \implies q) \iff (\sim p \vee q)$	$\sim (p \implies q) \iff (p \wedge \sim q)$

Rules of Inference

Modus Ponens	$p \implies q$ p $\therefore q$	Modus Tollens	$p \implies q$ $\sim q$ $\therefore \sim p$
Elimination	$p \vee q$ $\sim q$ $\therefore p$	Transitivity	$p \implies q$ $q \implies r$ $\therefore p \implies r$
Generalization	$p \implies p \vee q$ $q \implies p \vee q$	Specialization	$p \wedge q \implies p$ $p \wedge q \implies q$
Conjunction	p q $\therefore p \wedge q$	Contradiction Rule	$\sim p \implies F$ $\therefore p$

© 2011 B.E.Shapiro for integral-table.com. This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License. Revised July 20, 2011.

Figure: *Official Table of Logical Equivalences Cheat Sheet — [Source: PDF 3, Formula Sheet]*



List of rules of inference 1

List of rules of inference

This is a list of rules of inference, logical laws that relate to mathematical formulae.

Introduction

Rules of inference are syntactical **transform** rules which one can use to infer a conclusion from a premise to create an argument. A set of rules can be used to infer any valid conclusion if it is complete, while never inferring an invalid conclusion, if it is sound. A sound and complete set of rules need not include every rule in the following list, as many of the rules are redundant, and can be proven with the other rules.

Discharge rules permit inference from a subderivation based on a temporary assumption. Below, the notation

$$\varphi \vdash \psi$$

indicates such a subderivation from the temporary assumption φ to ψ .

Rules for classical sentential calculus

Sentential calculus is also known as propositional calculus.

Rules for negations

Reductio ad absurdum (or *Negation Introduction*)

$$\frac{\varphi \vdash \psi \qquad \varphi \vdash \neg\psi}{\neg\varphi}$$

Reductio ad absurdum (related to the law of excluded middle)

$$\frac{\neg\varphi \vdash \psi \qquad \neg\varphi \vdash \neg\psi}{\varphi}$$

Noncontradiction (or *Negation Elimination*)

$$\frac{\varphi \qquad \neg\varphi}{\psi}$$

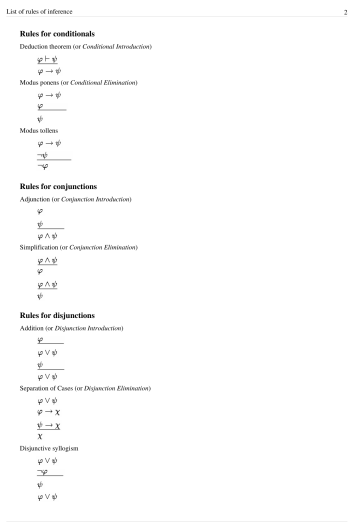
Double negation elimination

$$\frac{\neg\neg\varphi}{\varphi}$$

Double negation introduction

$$\frac{\varphi}{\neg\neg\varphi}$$

Figure: *Official Rules of Inference Derivations (Page 1) — [Source: PDF 2, Page 1]*



List of rules of inference 2

Rules for conditionals

Deduction theorem (or *Conditional Introduction*)

$$\frac{\varphi \vdash \psi}{\varphi \to \psi}$$

Modus ponens (or *Conditional Elimination*)

$$\frac{\varphi \to \psi \qquad \varphi}{\psi}$$

Modus tollens

$$\frac{\varphi \to \psi \qquad \neg\psi}{\neg\varphi}$$

Rules for conjunctions

Adjunction (or *Conjunction Introduction*)

$$\frac{\varphi \qquad \psi}{\varphi \wedge \psi}$$

Simplification (or *Conjunction Elimination*)

$$\frac{\varphi \wedge \chi}{\varphi} \qquad \frac{\varphi \wedge \chi}{\psi}$$

Rules for disjunctions

Addition (or *Disjunction Introduction*)

$$\frac{\varphi}{\varphi \vee \psi} \qquad \frac{\psi}{\varphi \vee \psi}$$

Separation of Cases (or *Disjunction Elimination*)

$$\frac{\varphi \vee \psi \qquad \varphi \to \chi \qquad \psi \to \chi}{\chi}$$

Disjunctive syllogism

$$\frac{\varphi \vee \psi \qquad \neg\varphi}{\psi} \qquad \frac{\varphi \vee \psi \qquad \neg\psi}{\varphi}$$

Figure: *Official Rules of Inference Derivations (Page 2) — [Source: PDF 2, Page 2]*

Module 7: Master AI Glossary (36 Key Definitions)

A curated, alphabetical lexicon of all 36 critical exam definitions spanning all 8 syllabus topics:

Term	Category	Official Definition & Exam Takeaway	Mathematical Formula / Notation
A* Search Algorithm	Search	Informed search evaluating f(n) = g(n) + h(n), balancing accumulated cost with heuristic estimate. Optimal if admissible/consistent.	$f(n) = g(n) + h(n)$
Admissibility	Search	A heuristic property meaning h(n) never overestimates the true cost to reach the goal: 0 ≤ h(n) ≤ h*(n). Guarantees tree A* optimality.	$0 \leq h(n) \leq h^*(n)$
Agent Function	Agents	The abstract mathematical mapping f: P* → A from complete percept histories to external actions executed via actuators.	$f : P^* \rightarrow A$
AI Winter	History	Periods of severe funding freezes in AI (1974-1980, 1987-1993) caused by over-promising and combinatorial explosion bottlenecks.	Over-promising → Funding freeze
Artificial Intelligence	Foundations	Computational engineering and science devoted to constructing rational agents capable of perceiving, reasoning, and acting.	AI = Cognitive Science + Agent Engineering
Backward Chaining	KR	Goal-driven reasoning: starts from a query hypothesis, finds matching rules, and recursively verifies sub-goals (used in MYCIN and Prolog).	Goal → Match Conclusions → Sub-goals
Consistency	Search	Triangle inequality for heuristics: h(n) ≤ c(n,a,n') + h(n'). Guarantees graph A* optimality and monotonic f-cost progression.	$h(n) \leq c(n,a,n') + h(n')$
Contradiction	Logic	A compound logical formula that evaluates to False under every possible truth assignment.	$P \wedge \neg P \equiv 0$ (False)
Dartmouth Workshop	History	1956 summer workshop organized by McCarthy, Minsky, Shannon, Rochester where 'Artificial Intelligence' was officially coined.	Birth of AI discipline (1956)
Declarative Knowledge	KR	Knowledge representing 'knowing that' — static assertions, facts, axioms, and relationships without execution control.	Declarative = 'Knowing That'
Deduction Theorem	Logic	Metalogical theorem establishing that premise P derives statement Q if and only if P → Q is a logical tautology.	$P \blacksquare Q \blacksquare \blacksquare (P \rightarrow Q)$
DIKW Pyramid	Foundations	Cognitive hierarchy: Data (raw symbols) → Information (context) → Knowledge (actionable rules) → Wisdom (evaluative principles).	Data → Info → Knowledge → Wisdom
Episodic Environment	Agents	An environment where the agent's experience is split into independent atomic episodes; current choices do not impact future episodes.	Episode i independent of Episode i+1
Forward Chaining	KR	Data-driven reasoning: starts with working memory facts, matches rule conditions, fires rules, and asserts conclusions until goal is found.	Facts → Match Premises → Fire Rules
Frames & Slots	KR	Minsky's data structure for stereotypical situations, comprising named Slots (attributes) filled with Fillers and default values.	Frame = { Slot: Value, Demon }
Goal-Based Agent	Agents	Agent architecture combining internal state with explicit goal representations, employing search and planning to reach goals.	State + Goal Formulation → Action
Greedy Best-First	Search	Heuristic search algorithm expanding the node with lowest heuristic value f(n) = h(n). Incomplete and non-optimal.	$f(n) = h(n)$
Heuristic Function	Search	An informed estimate of the cheapest path cost from node n to the nearest goal, derived from problem relaxations.	$h(n) \approx h^*(n)$ [True cost]

IDA*	Memory	Iterative Deepening A*: adapts IDDFS by using f-cost cutoffs instead of depth. Uses linear space O(b·d).	**Space: O(b · d) | f-limit cutoff**
Kowalski's Equation	KR	Theoretical foundation of logic programming: an algorithm consists of a logic component and a control execution component.	**Algorithm = Logic + Control**
Learning Agent	Agents	Modular agent architecture split into 4 elements: Learning Element, Performance Element, Critic, and Problem Generator.	**Learning + Performance + Critic + Explorer**
Model-Based Agent	Agents	Reflex agent maintaining an internal state estimating unobserved aspects of the world to overcome partial observability.	**State + Percept + Model → New State**
Modus Ponens	Logic	Propositional inference rule: from premises p and p → q, deduce conclusion q.	**p, p → q ■ q**
Modus Tollens	Logic	Propositional inference rule: from premises ¬q and p → q, deduce conclusion ¬p.	**¬q, p → q ■ ¬p**
PEAS Framework	Agents	Task environment specification: Performance measure (P), Environment (E), Actuators (A), and Sensors (S).	**P = Perf, E = Env, A = Act, S = Sens**
Procedural Knowledge	KR	Knowledge representing 'knowing how' — step-by-step algorithms, instructions, and executable recipes.	**Procedural = 'Knowing How'**
Production System	KR	Knowledge architecture composed of Working Memory (facts), Rule Base (IF-THEN), and Inference Engine (match-resolve-act).	**Working Memory + Rules + Engine**
Rationality	Foundations	Selecting actions expected to maximize performance given percept sequence to date and built-in knowledge. Distinct from omniscience.	**Max E[U] given Percepts P***
RBFS	Memory	Recursive Best-First Search: linear space O(b·d) search keeping track of the best alternative f-cost at each ancestor.	**Space: O(b · d) | Tracks alternative f**
Refractory Strategy	KR	Conflict resolution rule: prevents the same rule from firing twice on the identical piece of working memory data.	**Prevents infinite loops**
Resolution Principle	Logic	Automated theorem proving rule: from clauses (p ∨ q) and (¬p ∨ r), derive the resolvent (q ∨ r).	**(p ∨ q) ∧ (¬p ∨ r) ■ (q ∨ r)**
Semantic Network	KR	Knowledge graph where nodes represent concepts and directed edges represent relationships, primarily IsA and Instance links.	**Nodes = Concepts | Edges = IsA / Instance**
Simple Reflex Agent	Agents	Agent selecting actions based exclusively on current percept using condition-action rules. Has no memory of past percepts.	**condition → action**
SMA*	Memory	Simplified Memory-Bounded A*: uses all memory buffer M. Drops worst leaf and backs up its value to parent when full.	**Space: O(M) | Complete if M ≥ d**
Tautology	Logic	A compound logical formula that evaluates to True under all possible truth interpretations.	**P ∨ ¬P ≡ 1 (True)**
Thrashing	Memory	Phenomenon in RBFS where the algorithm repeatedly switches between competing branches, recalculating forgotten subtrees.	**Frequent backtracking + re-exploration**

Module 8: 20-Question Comprehensive Exam Simulation

A rigorous self-assessment test covering all syllabus topics with marked answer keys and detailed pedagogical rationales:

1. Who coined the term 'Artificial Intelligence' at the 1956 Dartmouth Workshop?

- A. Alan Turing
- ✓ **B. John McCarthy**
- C. Marvin Minsky
- D. Claude Shannon

Rationale: *John McCarthy coined 'Artificial Intelligence' in his 1955 proposal for the 1956 Dartmouth College workshop.*

2. In the PEAS framework, what does the acronym stand for?

- A. Program, Environment, Algorithm, Sensors
- ✓ **B. Performance, Environment, Actuators, Sensors**
- C. Process, Environment, Architecture, Storage
- D. Percept, Execution, Action, Strategy

Rationale: *PEAS = Performance measure, Environment, Actuators, Sensors.*

3. What is the evaluation function for the A* search algorithm?

- A. $f(n) = h(n)$
- B. $f(n) = g(n)$
- ✓ **C. $f(n) = g(n) + h(n)$**
- D. $f(n) = g(n) \times h(n)$

Rationale: *A* combines the exact path cost from start g(n) with the heuristic estimate to the goal h(n).*

4. An admissible heuristic h(n) is formally defined as one that...

- A. Always overestimates true cost
- ✓ **B. Never overestimates true cost: $0 \leq h(n) \leq h^*(n)$**
- C. Equals 0 for all nodes
- D. Is strictly greater than true cost

Rationale: *Admissibility means h(n) is an optimistic estimate that never overestimates the true cost h*(n) to the goal.*

5. Which inference rule derives 'q' from premises 'p' and 'p → q'?

- A. Modus Tollens
- ✓ **B. Modus Ponens**
- C. Resolution
- D. Hypothetical Syllogism

Rationale: *Modus Ponens (affirming the antecedent): If p is true and p implies q, then q must be true.*

6. Which search algorithm utilizes strictly linear space O(b·d)?

- A. Standard A* Search
- ✓ **B. Iterative Deepening A* (IDA*)**
- C. Breadth-First Search (BFS)
- D. SMA* with unbounded memory

Rationale: *IDA* runs a sequence of depth-first searches bounded by f-cost cutoffs, requiring only linear space O(b·d).*

7. In a fully observable environment, an intelligent agent...

- A. Requires internal memory to infer unseen objects
- ✓ **B. Has complete state access at all times through sensors**
- C. Must continuously learn rules
- D. Has noise in every step

Rationale: *Full observability gives complete, noiseless access to the current state, so no internal world model is required.*

8. Forward chaining is characterized as being...

- ✓ **A. Data-driven, reasoning forward from facts to conclusions**
- B. Goal-driven, reasoning backward from hypothesis
- C. Exclusively probabilistic
- D. Unable to resolve conflicts

Rationale: *Forward chaining starts with known facts in working memory and triggers rules forward to derive new facts.*

9. According to De Morgan's Laws, what is the logical equivalence of ¬(p ∧ q)?

- A. ¬p ∧ ¬q
- ✓ **B. ¬p ∨ ¬q**
- C. p ∨ q
- D. ¬p → ¬q

Rationale: *¬(p ∧ q) ≡ ¬p ∨ ¬q — the negation flips the conjunction into a disjunction.*

10. When memory is full in SMA*, which node does the algorithm drop?

- A. The root node
- B. The deepest node
- ✓ **C. The shallowest, highest f-cost (worst) leaf node**
- D. A randomly selected node

Rationale: *SMA* discards the shallowest, highest f-cost leaf node and backs up its value to the parent node.*

11. In Prolog, how is a rule formally structured?

- ✓ **A. Head :- Body.**
- B. Body -> Head.
- C. IF Condition THEN Action.
- D. ?- Query.

Rationale: *Prolog rules are Horn clauses written as 'Head :- Body.', meaning Head is true IF Body is true.*

12. What is the contrapositive of the conditional statement (p → q)?

- A. q → p
- B. ¬p → ¬q
- ✓ **C. ¬q → ¬p**
- D. ¬p ∨ q

Rationale: *The contrapositive of (p → q) is (¬q → ¬p), and both are strictly logically equivalent.*

13. Which environment dimension means current actions do not impact future decisions?

- A. Deterministic
- B. Static
- ✓ **C. Episodic**
- D. Discrete

Rationale: *In episodic environments, each interaction is independent; the agent's current action has no bearing on future episodes.*

14. What does Consistency (Monotonicity) in heuristics require?

- A. h(n) = 0 for all nodes
- ✓ **B. h(n) ≤ c(n,a,n') + h(n') [Triangle inequality]**
- C. h(n) > h*(n)
- D. f(n) = g(n)

Rationale: *Consistency requires h(n) ≤ c(n,a,n') + h(n'), guaranteeing that f-values along any path never decrease.*

15. What type of knowledge is described as 'knowing how' to execute procedures?

- A. Declarative Knowledge
- ✓ **B. Procedural Knowledge**
- C. Relational Knowledge
- D. Inferential Knowledge

Rationale: *Procedural knowledge ('knowing how') represents step-by-step algorithms, instructions, and executable rules.*

16. Which conflict resolution strategy avoids firing the same rule twice on identical data?

- A. Recency
- B. Specificity
- ✓ **C. Refractory**
- D. Priority

Rationale: *Refractory ensures that a rule cannot fire more than once on the exact same data, preventing infinite loops.*

17. Greedy Best-First Search prioritizes nodes using which evaluation function?

- A. f(n) = g(n)
- ✓ **B. f(n) = h(n)**
- C. f(n) = g(n) + h(n)
- D. f(n) = c(s,a,s')

Rationale: *Greedy Best-First search expands the node that looks closest to the goal based solely on the heuristic h(n).*

18. How many components comprise a Learning Agent architecture?

- A. 2 components
- B. 3 components
- ✓ **C. 4 components (Learning, Performance, Critic, Explorer)**
- D. 5 components

Rationale: *A Learning Agent consists of 4 parts: Learning Element, Performance Element, Critic, and Problem Generator.*

19. The Resolution Refutation principle proves a statement S by...

- A. Assuming S is true and verifying it
- ✓ **B. Negating S and deriving a contradiction (empty clause ■)**
- C. Converting S directly to DNF
- D. Using forward chaining only

Rationale: *Resolution refutation assumes ¬S and applies resolution until the empty clause ■ (contradiction) is derived.*

20. Robert Kowalski summarized logic programming with which famous equation?

- A. AI = Architecture + Program
- ✓ **B. Algorithm = Logic + Control**
- C. Search = Cost + Heuristic
- D. Knowledge = Information + Rules

Rationale: *Kowalski's famous formula states: Algorithm = Logic + Control.*